

PROCESAMIENTO DE DATOS MASIVOS

Actividad 1 – Programación en Hadoop

Ejercicio 1

Enunciado

Dado el dataset que se adjunta a la actividad, que contiene los resultados de los partidos de la primera división de fútbol española, **crea un programa llamado puntosEquipo que para cada equipo calcule el total de puntos conseguidos (victoria 3 puntos, empate 1 punto, derrota 0 puntos), siguiendo el formato equipo;puntos**. La salida debe mostrar los equipos ordenados alfabéticamente. Se valorará positivamente la optimización del programa, por ejemplo minimizando el número de información que se transmite por el cluster. Debe implementar map, combine y reduce.

Solución

Se propone la solución contenida en la carpeta “ej1”, compuesta por los siguientes documentos:

- Datos de entrada: resultados_futbol.csv
- Código: se han desarrollado los scripts mapper_puntosEquipo.py y reducer_puntosEquipo.py. Dado que el problema a resolver consiste en una suma, que cumple la propiedad conmutativa, el script reducer_puntosEquipo.py servirá como combiner y como reducer
- Comando de ejecución: el código a escribir en la terminal se reporta en terminal_puntosEquipo.txt
- Resultado: el resultado generado por el programa se reporta en la carpeta salidaPuntosEquipo

Esta solución se ha ejecutado utilizando la versión 3.4.0 de Hadoop en un único nodo.

Código mapper

El mapper recibe todos los partidos jugados en la liga española de fútbol entre las temporadas 2000-2001 y 2023-2024 y emite como salida dos resultados por partido en formato 'equipo;puntos'. Uno de los resultados se corresponde al equipo local y otro al equipo visitante. Por ejemplo:

- Input: si recibe las líneas
 - 20,2000-2001,3,0-2,Valencia - Alavés,Valencia,Alavés
 - 216,2000-2001,22,0-1,Alavés - Valencia,Alavés,Valencia
 - 482,2001-2002,11,1-1,Alavés - Valencia,Alavés,Valencia
 - 671,2001-2002,30,1-2,Valencia - Alavés,Valencia,Alavés
- Output:
 - Valencia;0
 - Alavés;3
 - Alavés;0
 - Valencia;3
 - Valencia;1

- Alavés;1
- Valencia;0
- Alavés;3

Para cada línea o partido, el código retiene la información relevante para realizar los cálculos (resultado, que está en el tercer elemento; equipo local, elemento 5; y equipo visitante, elemento 6). Después, calcula los puntos obtenidos por cada equipo comparando los goles marcados. Por último, emite los resultados en formato 'equipo;puntos', para que puedan ser procesados por el combiner.

```
import sys

'''
Mapper de puntosEquipo

input: cada línea del csv (partido)
output: se emiten 2 por partido
- local;ptos_local
- visitante;ptos_visitante
'''

# por cada línea (partido):
# 1) se guarda la información relevante (resultado, equipo local, eq.visitante)
# 2) se calculan los puntos obtenidos por cada equipo en ese partido
# 3) se emiten los pares (equipo, ptos) para cada equipo (2 res. por partido)

for linea in sys.stdin:
    linea = linea.strip()

    # Añadimos un bloque try-except para manejar líneas con formato incorrecto
    # como la línea de cabecera o cualquier otra con formato inesperado
    try: # guardar la información relevante
        linea_parse = linea.split(',')
        resultado = linea_parse[3]
        local = linea_parse[5]
        visitante = linea_parse[6]
        goles_local, goles_visit = resultado.split('-')
        goles_local = int(goles_local) # estaba como string
        goles_visit = int(goles_visit) # estaba como string
        # Normalizar datos
        if local == "Deportivo Alavés":
            local = "Alavés"
        elif local == "R. Sociedad":
            local = "Real Sociedad"
        if visitante == "Deportivo Alavés":
            visitante = "Alavés"
        elif visitante == "R. Sociedad":
            visitante = "Real Sociedad"
    except (IndexError, ValueError):
        continue
    # calcular los ptos obtenidos por cada equipo en el partido
    if goles_local > goles_visit:
        ptos_local = 3
        ptos_visit = 0
    elif goles_local < goles_visit:
        ptos_local = 0
        ptos_visit = 3
    else:
        ptos_local = 1
        ptos_visit = 1
    # emitir 2 pares: equipo\tptos
    print(f"{local};{ptos_local}")
    print(f"{visitante};{ptos_visit}")
```

Código combiner

```
#!/usr/bin/python3
import sys

'''
Cominer y Reducer de puntosEquipo
Input: equipo;ptos
Output: equipo;ptos
'''

subproblema = None
suma_ptos = 0

for claveValor in sys.stdin:
    equipo, ptos = claveValor.split('; ', 1) #había usado ; como separador
    ptos = int(ptos) # por defecto era string
    if subproblema == None: # primer equipo
        subproblema = equipo
    if subproblema == equipo: # si sigo con ese equipo, sumo
        suma_ptos += ptos
    else: # si llega un equipo diferente
        print("%s;%s" % (subproblema, suma_ptos)) # emito total de equipo
        # y paso al siguiente equipo
        subproblema = equipo
        suma_ptos = ptos
# emito el último equipo fuera del bucle
print("%s;%s" % (subproblema, suma_ptos))
```

El combiner recibe, en cada nodo, los resultados emitidos por el mapper. Suma los puntos obtenidos por cada equipo con los datos disponibles en el nodo y la devuelve un resultado por equipo en formato 'equipo;puntos'. Siguiendo el ejemplo anterior:

- Input:
 - Valencia;0
 - Alavés;3
 - Alavés;0
 - Valencia;3
 - Alavés;1
 - Valencia;1
 - Valencia;0
 - Alavés;3
- Output:
 - Valencia;4
 - Alavés;7

Para alcanzar la solución se parte el problema principal (calcular el total de puntos conseguidos por cada equipo) en subproblemas (para cada equipo, calcular el total de puntos conseguidos).

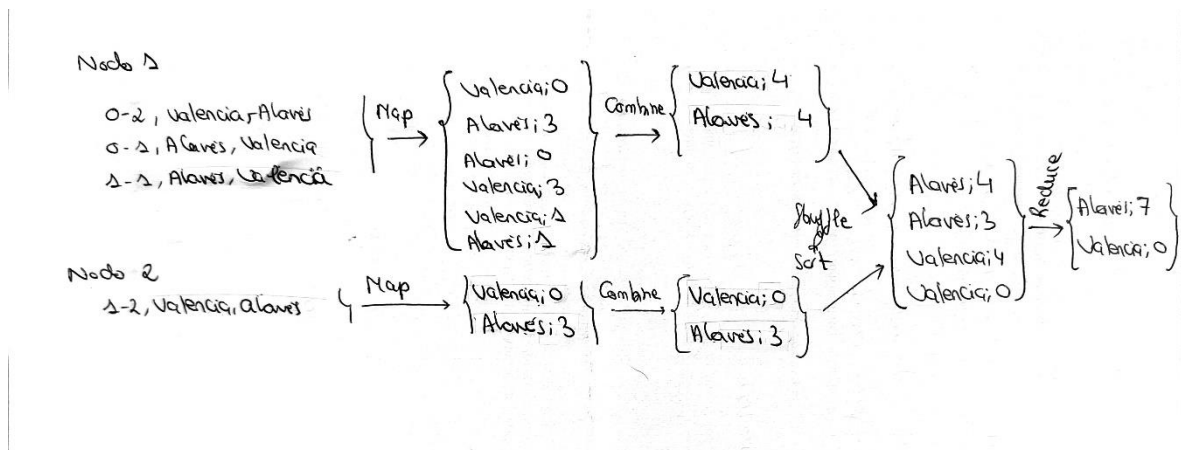
Observamos que gracias al combiner, estaremos escribiendo en disco y enviando al reducer menos datos que si simplemente utilizáramos el mapper. En nuestro ejemplo, pasaríamos de escribir y enviar 8 datos a 2.

Código reducer

Como el problema consiste en sumar puntos y la suma cumple las propiedades conmutativa y asociativa, utilizamos el mismo código para el combiner y para el reducer. Para ello, es importante que la entrada y la salida tengan el mismo formato, en este caso 'equipo;puntos'.

Al reducer le llegarán todos los resultados emitidos por todos los combiners en los diferentes nodos. Gracias al Shuffle and Sort, el reducer recibirá los datos de todos los nodos ordenados por clave alfabéticamente. Se sigue la lógica explicada anteriormente de dividir el problema en subproblemas, siendo cada subproblema la suma de puntos de un equipo.

El siguiente diagrama muestra el proceso si los datos de nuestro ejemplo estuvieran distribuidos en dos nodos:



Ejecución

Seguimos los siguientes pasos para ejecutar el programa:

1. Ubicarnos en la ruta deseada (cd /home/agorostizal/Desktop/practica1/ej1)
2. Dar permiso de ejecución a los scripts (chmod u+x mapper_puntosEquipo.py, chmod u+x reducer_puntosEquipo.py)
3. Ejecutar el programa:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar
-files ./mapper_puntosEquipo.py,./reducer_puntosEquipo.py
-mapper ./mapper_puntosEquipo.py
-reducer ./reducer_puntosEquipo.py
-combiner ./reducer_puntosEquipo.py
-input resultados_futbol.csv
-output ./salidaPuntosEquipo
```

```

agorostizal@vbox: ~/Desktop/practica1/ej1
File Edit View Search Terminal Help
[agorostizal@vbox ~]$ cd /home/agorostizal/Desktop/practica1/ej1
[agorostizal@vbox ej1]$ chmod u+x mapper_puntosEquipo.py
[agorostizal@vbox ej1]$ chmod u+x reducer_puntosEquipo.py
[agorostizal@vbox ej1]$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.4.0.jar -
files ./mapper_puntosEquipo.py,./reducer_puntosEquipo.py -mapper ./mapper_puntosEquipo.py -reducer
./reducer_puntosEquipo.py -combiner ./reducer_puntosEquipo.py -input resultados_futbol.csv -output
./salidaPuntosEquipo

```

Y obtenemos la siguiente salida en terminal:

```
File System Counters
  FILE: Number of bytes read=1428610
  FILE: Number of bytes written=1758372
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
Map-Reduce Framework
  Map input records=9121
  Map output records=18240
  Map output bytes=232674
  Map output materialized bytes=711
  Input split bytes=118
  Combine input records=18240
  Combine output records=42
  Reduce input groups=42
  Reduce shuffle bytes=711
  Reduce input records=42
  Reduce output records=42
  Spilled Records=84
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=4
  Total committed heap usage (bytes)=343932928
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=569436
File Output Format Counters
  Bytes Written=637
2025-06-28 17:29:52,631 INFO streaming.StreamJob: Output directory: ./salidaPuntosEquipo
```

- El mapper ha recibido 9.121 filas y ha devuelto 18.240 (dos resultados por fila, excepto para la primera, ya que corresponde a los encabezados).
- El combiner ha recibido esos 18.240 datos, los ha combinado y ha devuelto 42 (uno por equipo).
- El reducer ha recibido esos 42 datos y ha emitido 42. Esto se debe a que se ha trabajado en un único nodo, por lo que no se ha aprovechado el potencial del combiner.

Los resultados se han escrito en la carpeta 'salidaPuntosEquipo', disponible entre los materiales entregados en este ejercicio.

```
Alavés;498
Albacete;75
Almería;283
Athletic;1242
Atlético;1463
Barcelona;1965
Celta;886
Cádiz;161
Córdoba;20
Deportivo;883
Eibar;302
Elche;184
Espanyol;1055
Getafe;840
```

Gimnàstic Tarragona;28
Girona;137
Granada;359
Huesca;67
Hércules;35
Las Palmas;191
Leganés;159
Levante;583
Mallorca;825
Málaga;791
Numancia;148
Osasuna;865
Racing;524
Rayo Vallecano;525
Real Betis;989
Real Madrid;1959
Real Murcia;56
Real Oviedo;86
Real Sociedad;1106
Real Sporting;237
Real Valladolid;587
Real Zaragoza;551
Recreativo;167
Sevilla;1386
Tenerife;74
Valencia;1449
Villarreal;1280
Xerez;34